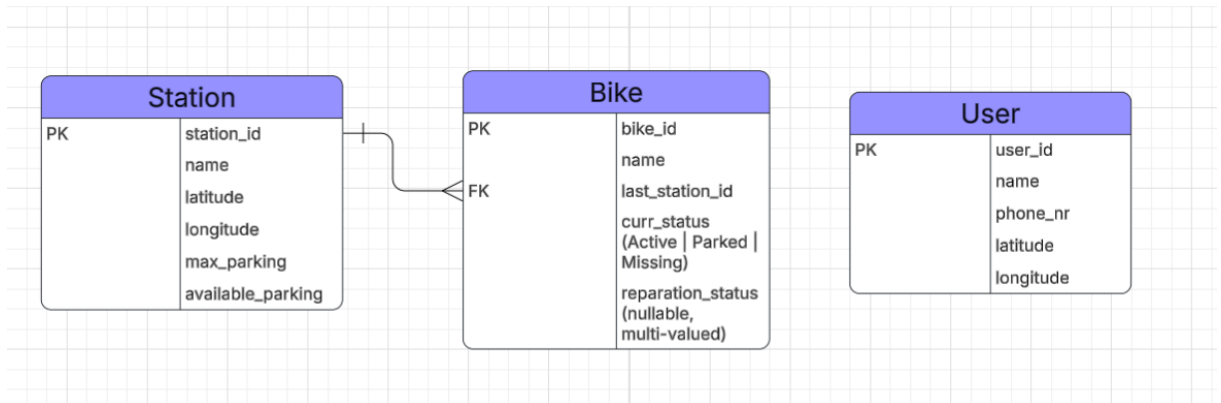


Obligatory Assignment 1 – INF115 – Jon Hauge

Problem 1

1.



2.

No, all three tables do not participate in a direct relationship. There is a relationship between the *Station* and *Bike* tables, but no direct relationship between *Bike* and *User*. In practice, this means all bikes are associated with stations, while the users of the bikes have no ownership over them nor the stations. The only relation between *User* and the rest is an indirect relationship through filed complaints with *Bike*, but this is not shown in the current normal form of the diagram.

Problem 2

1.

The problem with the proposed solution is incorrectly handling functional dependencies. The junior developer wants to add the five attributes of the subscription directly to the table *User*. Firstly, this violates the atomic requirement of 1NF, since we would have no way of knowing whether users have more than one membership without multiple columns, storing arrays or other such methods.

Secondly, having the *SubscriptionID* attribute in the same table as the primary key *UserID* would create a transitive dependency, violating 3NF.

Thirdly, mixing users with subscriptions violates good DBMS principles by intertwining people with time-based contracts. This would lead to unwanted dependencies, such as losing historical subscription data by deleting a user. Lastly, though minor, the wording on the *Start* and *End* attributes make it unclear whether this should be a time of day, date or another format.

2.

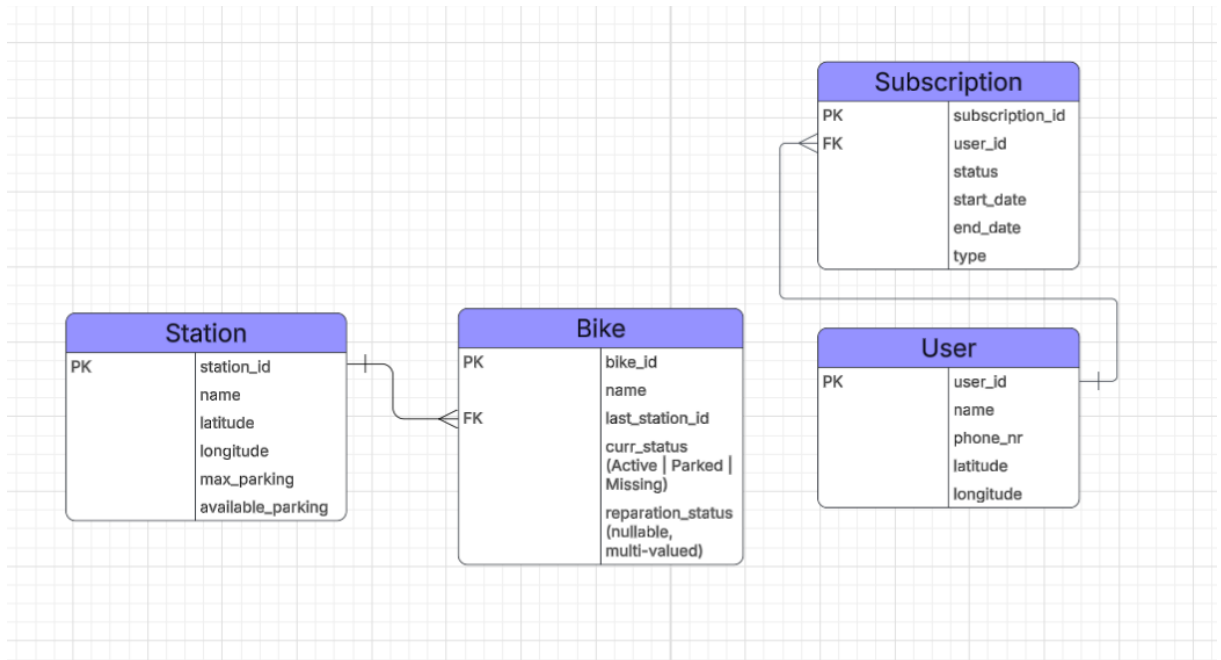
The better solution to the problem is to separate the proposed attributes to another entity, *Subscription*, which includes all the proposed attributes plus *UserID* and changes the *Start* and *End* attributes to *StartDate* and *EndDate* for clarity. The primary key of this table will be the *SubscriptionID* attribute. The one foreign key will be *UserID* in *Subscription* which will have a direct many-to-one relationship with the *UserID* in the *User* table.

This means adding only one new table, *Subscriptions*:

- Subscriptions (#SubscriptionID, *UserID, Status, StartDate, EndDate, Type)
- User (#UserID, Name, PhoneNr, Latitude, Longitude)

FOREIGN KEY (UserID) REFERENCES User(UserID)

3.



Problem 3

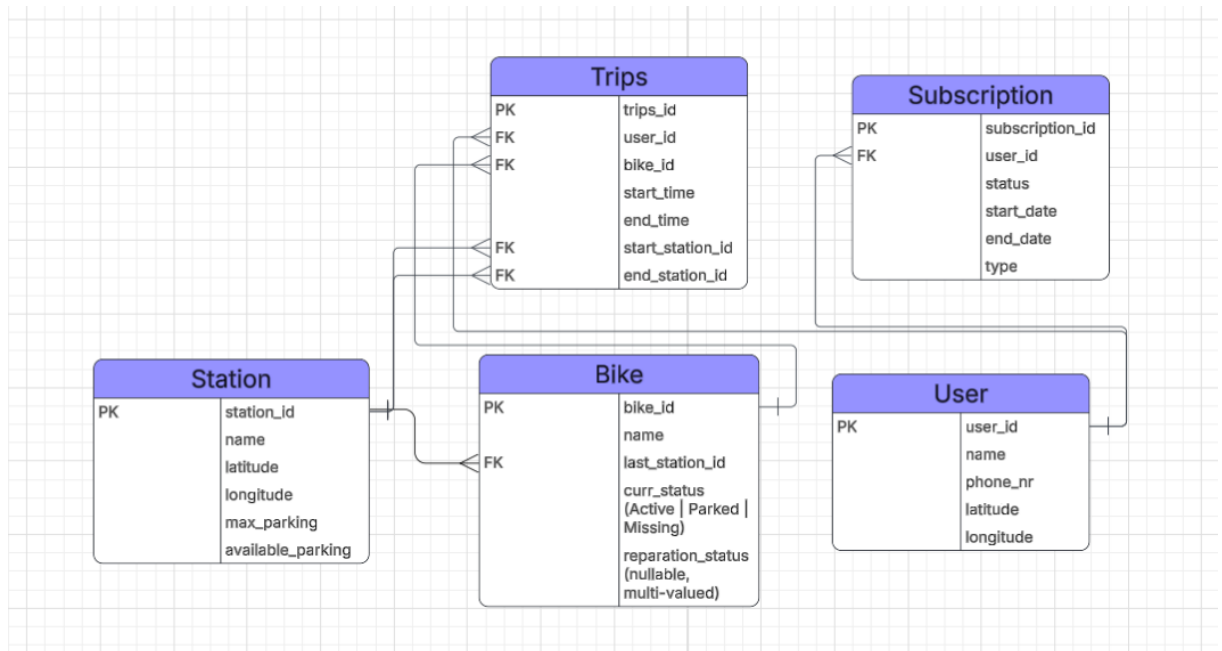
a)

The attributes needed for a *Trip* entity will be *TripID*, *UserID*, *BikeID*, *StartTime*, *EndTime*, *StartStationID* and *EndStationID*. The primary key of this table will be the *TripID* attribute, as it is unique to every trip, while *UserID*, *BikeID*, *StartStationID* and *EndStationID* will be foreign keys with many-to-one relationships to the respective attributes in the *User*, *Bike* and *Station* tables. The *Trips* table will look like this:

- Trips (#TripID, *UserID, *BikeID, StartTime, EndTime, *StartStationID, *EndStationID)
- Bike (#BikeID, Name, *LastStationID, CurrStatus (Active | Parked | Missing), ReparationStatus (nullable, multi-valued))
- User (#UserID, Name, PhoneNr, Latitude, Longitude)
- Station (#StationID, Name, Latitude, Longitude, MaxParking, AvailableParking)

FOREIGN KEY (UserID) REFERENCES User (UserID),
 FOREIGN KEY (BikeID) REFERENCES Bike (BikeID),
 FOREIGN KEY (StartStationID) REFERENCES Station (StationID),
 FOREIGN KEY (EndStationID) REFERENCES Station (StationID) and
 FOREIGN KEY (LastStationID) REFERENCES Station (StationID)

b)



c)

There are four normal forms we can normalize database design to: 1NF, 2NF, 3NF and BCNF. All of them stack, meaning a table in 3NF also satisfies the criteria for 1NF and 2NF.

1NF – The first normal form in normalization requires all the values of a table to be atomic. This means composite values, such as having first- and last-names in one column (“tables in tables”), are disallowed. To normalize a table to 1NF, all composite values must be decomposed and given their own column. If this is already true before any normalization, the table was already in 1NF.

2NF – The second normal form requires all values in the table to not contain attributes that are partially dependent on the primary key and also to be in 1NF. This means composite primary keys are disallowed. If a table has a partial dependence, where an attribute is only partly dependent on the candidate key, it must be split into two or more tables such that all partial dependences are removed.

3NF – The third normal form requires that the table contains no transitive functional dependencies and also be in 2NF. This means functional dependencies, where a non-candidate key attribute has a dependency on another non-candidate key attribute, are disallowed. To normalize to 3NF, the table will need to be split up until no transitive dependencies remain.

BCNF – The Boyce-Codd normal form requires that any minimal determinant is a candidate key. This is similar to 3NF, but there are examples of 3NF that breach BCNF. These breaches assume two composite and overlapping candidate keys, i.e. every determinant attribute must be a superkey in BCNF. An example of this is: Example (Student, Course, Instructor). The functional dependencies here state that *Student* and *Course* determine *Instructor* and therefore all attributes, making them a composite candidate key. The table contains no transitive dependencies and satisfies 2NF, making it 3NF. However, the *Instructor* also determines the *Course*, meaning that not every determinant is a superkey. This breaches BCNF.

d)

The current database now contains three tables, *Bike*, *User* and *Station*, and two entities, *Subscription* and *Trips*. Because we already planned for it when implementing the entities, these already satisfy BCNF. The changes we need to make are only to the tables themselves.

Bike – To get this table to BCNF, we need to make sure all determinant attributes are superkeys. As it is implemented now, that is not the case, as both the current status and reparation status attributes of the bike determine

other attributes of the table, such as the 12 different complaint types. To normalize, we will need to move these attributes into separate entities that we name *Complaint* and *BikeStatus*.

The *Complaint* entity will have a *ComplaintID* attribute as a primary key, with foreign keys of both the *BikeID* and *UserID* attributes, making the indirect relationship between the *Bike* and *User* tables show in the ER-diagram. We will also need another foreign key to a new, different entity, *ComplaintType*, which will have its primary key as *ComplaintTypeID* and a *Name* attribute for the 12 different complaints. This is because having these in *Complaint* would breach the atomic requirement of 1NF. Lastly, we can also add a *DateReported* attribute for the *Complaint* entity to finish it out.

The *BikeStatus* entity is more straight forward, as we only need a primary key *StatusID* and a *Name* attribute. The *StatusID* will also be added as a foreign key in the *Bike* table, replacing the *CurrStatus* attribute. The *ReparationStatus* attribute can be fully removed from *Bike*, since complaints are handled through the *BikeID*.

User – To get this table into BCNF, we only need to create a many-to-one relationship between the primary key *UserID* and the foreign key *UserID* in the *Complaint* entity. It may be possible that *PhoneNr* is a determinant for *UserID*, meaning it would need its own entity, but in this task, I assume that phone numbers are not unique and can be shared with e.g. family members.

Station – This table is already in BCNF, but there is another change that should be made. As it stands, every determinant is a superkey, but the *AvailableParking* attribute is derived. Since our job is to reduce redundancy, a smart move is to remove the *AvailableParking* attribute and rather calculate that through *MaxParking* when needed, as the one-to-many link between the primary key *StationID* and the foreign key *LastStationID* in *Bike* gives us the knowledge over which and how many bikes are at each station.

With these changes done, we end up with these tables:

- Bike (#BikeID, Name, *LastStationID, *StatusID)
- User (#UserID, Name, PhoneNr, Latitude, Longitude)

- Station (#StationID, Name, Latitude, Longitude, MaxParking)
- Subscriptions (#SubscriptionID, *UserID, Status, StartDate, EndDate, Type)
- Trips (#TripID, *UserID, *BikeID, StartTime, EndTime, *StartStationID, *EndStationID)
- Complaint (#ComplaintID, *BikeID, *UserID, *ComplaintTypeID, DateReported)
- ComplaintType(#ComplaintTypeID, Name)
- BikeStatus (#StatusID, StatusName)

FOREIGN KEY (LastStationID) REFERENCES Station (StationID),
FOREIGN KEY (StatusID) REFERENCES BikeStatus (StatusID),
FOREIGN KEY (UserID) REFERENCES User (UserID),
FOREIGN KEY (BikeID) REFERENCES Bike (BikeID),
FOREIGN KEY (StartStationID) REFERENCES Station (StationID),
FOREIGN KEY (EndStationID) REFERENCES Station (StationID) and
FOREIGN KEY (ComplaintTypeID) REFERENCES ComplaintType (ComplaintTypeID)

Problem 4

1.

```
import sqlite3

con = sqlite3.connect("bysykkel.db")
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON;")

# Creating tables

# Bike
bike_query = "CREATE TABLE IF NOT EXISTS Bike(" \
             "BikeID INTEGER, " \
             "Name VARCHAR NOT NULL, " \
             "LastStationID INTEGER, " \
             "StatusID INTEGER, " \
```

```
        "PRIMARY KEY (BikeID), " \
        "FOREIGN KEY (LastStationID) REFERENCES Station (StationID), "
\
        "FOREIGN KEY (StatusID) REFERENCES BikeStatus (StatusID));"

# User
user_query = "CREATE TABLE IF NOT EXISTS User(" \
    "UserID INTEGER, " \
    "Name VARCHAR NOT NULL, " \
    "PhoneNr VARCHAR, " \
    "Latitude FLOAT(10,6), " \
    "Longitude FLOAT(10,6), " \
    "PRIMARY KEY (UserID));"

# Station
station_query = "CREATE TABLE IF NOT EXISTS Station(" \
    "StationID INTEGER UNIQUE NOT NULL, " \
    "Name VARCHAR NOT NULL, " \
    "Latitude FLOAT(10,6), " \
    "Longitude FLOAT(10,6), " \
    "MaxParking SMALLINT NOT NULL, " \
    "PRIMARY KEY (StationID));"

# Subscriptions
subscriptions_query = "CREATE TABLE IF NOT EXISTS Subscriptions(" \
    "SubscriptionID INTEGER, " \
    "UserID INTEGER NOT NULL, " \
    "Status VARCHAR NOT NULL, " \
    "StartDate VARCHAR NOT NULL, " \
    "EndDate VARCHAR, " \
    "Type VARCHAR NOT NULL, " \
    "PRIMARY KEY (SubscriptionID), " \
    "FOREIGN KEY (UserID) REFERENCES User (UserID));"

# Trips
trips_query = "CREATE TABLE IF NOT EXISTS Trips(" \
    "TripID INTEGER, " \
    "UserID INTEGER NOT NULL, " \
    "BikeID INTEGER NOT NULL, " \
    "StartTime VARCHAR NOT NULL, " \
    "EndTime VARCHAR, " \
    "StartStationID INTEGER NOT NULL, " \
    "EndStationID INTEGER, " \
    "PRIMARY KEY (TripID), " \
    "FOREIGN KEY (UserID) REFERENCES User (UserID), " \
    "FOREIGN KEY (BikeID) REFERENCES Bike (BikeID), " \
    "FOREIGN KEY (StartStationID) REFERENCES Station (StationID),
" \
    "FOREIGN KEY (EndStationID) REFERENCES Station (StationID));"
```

```
# Complaint
complaint_query = "CREATE TABLE IF NOT EXISTS Complaint(" \
    "ComplaintID INTEGER, " \
    "BikeID INTEGER NOT NULL, " \
    "UserID INTEGER NOT NULL, " \
    "ComplaintTypeID INTEGER NOT NULL, " \
    "DateReported VARCHAR NOT NULL, " \
    "PRIMARY KEY (ComplaintID), " \
    "FOREIGN KEY (BikeID) REFERENCES Bike (BikeID), " \
    "FOREIGN KEY (UserID) REFERENCES User (UserID), " \
    "FOREIGN KEY (ComplaintTypeID) REFERENCES ComplaintType"
(ComplaintTypeID));"

# Parts of the following code was formulated / helped by OpenAI's ChatGPT
# This includes the use of autoincrementing for the complaint_type and
bike_status
# All code helped by AI is understood in and of itself and in relation to the
rest of the program
# OpenAI - https://chatgpt.com - retrieved 01.03.2026

# ComplaintType
complaint_type_query = "CREATE TABLE IF NOT EXISTS ComplaintType(" \
    "ComplaintTypeID INTEGER PRIMARY KEY AUTOINCREMENT , " \
    "Name VARCHAR UNIQUE NOT NULL);"

# BikeStatus
bike_status_query = "CREATE TABLE IF NOT EXISTS BikeStatus(" \
    "StatusID INTEGER PRIMARY KEY AUTOINCREMENT , " \
    "StatusName VARCHAR UNIQUE NOT NULL);"

# Executing queries
cur.execute(station_query)
cur.execute(bike_status_query)
cur.execute(user_query)
cur.execute(bike_query)
cur.execute(subscriptions_query)
cur.execute(trips_query)
cur.execute(complaint_type_query)
cur.execute(complaint_query)

# Committing changes
con.commit()
con.close()

print("Database changes were successfull!")
```

2.

```
import sqlite3
import csv

con = sqlite3.connect("bysykkel.db")
cur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON;")
cur.execute("PRAGMA journal_mode = WAL;")
cur.execute("PRAGMA synchronous = OFF;")

# Importing bysykkel.csv

# Parts of the following code was formulated / helped by OpenAI's ChatGPT
# This includes the idea of using dicts/sets to import data and the
# code for the status-map / new_bikes.
# All code helped by AI is understood in and of itself and in relation to the
# rest of the program
# OpenAI - https://chatgpt.com - retrieved 01.03.2026

stations = {}
users = {}
bikes = {}
subscriptions = {}
trips = {}
statuses = set()

with open("bysykkel.csv", newline='', encoding="utf-8") as csvfile:
    reader = csv.DictReader(csvfile)

    for row in reader:

        if row["start_station_id"]:
            sid = int(row["start_station_id"])
            stations[sid] = (sid,
                             row["start_station_name"],
                             float(row["start_station_latitude"]) if
row["start_station_latitude"] else None,
                             float(row["start_station_longitude"]) if
row["start_station_longitude"] else None,
                             int(row["start_station_max_spots"]) if
row["start_station_max_spots"] else None)

        if row["end_station_id"]:
            sid = int(row["end_station_id"])
            stations[sid] = (sid,
```

```
        row["end_station_name"],
        float(row["end_station_latitude"]) if
row["end_station_latitude"] else None,
        float(row["end_station_longitude"]) if
row["end_station_longitude"] else None,
        int(row["end_station_max_spots"]) if
row["end_station_max_spots"] else None)

    if row["user_id"]:
        uid = int(row["user_id"])
        users[uid] = (uid,
                      row["user_name"],
                      row["user_phone_number"],
                      None, None)

    statuses.add(row["bike_status"])

    if row["bike_id"]:
        bid = int(row["bike_id"])
        bikes[bid] = (bid,
                      row["bike_name"],
                      int(row["bike_station_id"]) if row["bike_station_id"]
else None,
                      row["bike_status"])

    if row["subscription_id"]:
        sid = int(row["subscription_id"])
        subscriptions[sid] = (sid,
                              int(row["user_id"]) if row["user_id"] else
None,
                              "Active",
                              row["subscription_start_time"],
                              None,
                              row["subscription_type"])

    if row["trip_id"]:
        tid = int(row["trip_id"])
        trips[tid] = (tid,
                      int(row["user_id"]) if row["user_id"] else None,
                      int(row["bike_id"]) if row["bike_id"] else None,
                      row["trip_start_time"],
                      row["trip_end_time"] or None,
                      int(row["start_station_id"]) if
row["start_station_id"] else None,
                      int(row["end_station_id"]) if row["end_station_id"]
else None)

with con:
```

```
cur.executemany("INSERT INTO Station(" \
                "StationID, Name, Latitude, Longitude, MaxParking) " \
                "VALUES (?, ?, ?, ?, ?)", stations.values())

cur.executemany("INSERT INTO User(" \
                "UserID, Name, PhoneNr, Latitude, Longitude) " \
                "VALUES (?, ?, ?, ?, ?)", users.values())

cur.executemany("INSERT INTO BikeStatus (StatusName) "
                "VALUES (?)", [(status,) for status in statuses])

cur.execute("SELECT StatusID, StatusName FROM BikeStatus")
status_map = {name : sid for sid, name in cur.fetchall()}

new_bikes = [(b[0], b[1], b[2], status_map[b[3]]) for b in bikes.values()]

cur.executemany("INSERT INTO Bike(" \
                "BikeID, Name, LastStationID, StatusID) " \
                "VALUES (?, ?, ?, ?)", new_bikes)

cur.executemany("INSERT INTO Subscriptions (" \
                "SubscriptionID, UserID, Status, StartDate, EndDate, Type) " \
                "VALUES (?, ?, ?, ?, ?, ?)", subscriptions.values())

cur.executemany("INSERT INTO Trips(" \
                "TripID, UserID, BikeID, StartTime, EndTime, " \
                "StartStationID, EndStationID) " \
                "VALUES (?, ?, ?, ?, ?, ?, ?)", trips.values())

con.close()
print("Imported data successfully")
```

3.

```
import sqlite3

con = sqlite3.connect("bysykkel.db")
cur = con.cursor()

sub_type_query = "SELECT Type, Count(*) as PurchasedSubs " \
                 "FROM Subscriptions " \
                 "WHERE Type IN ('Year', 'Month', 'Week', 'Day') " \
                 "GROUP BY Type " \
                 "ORDER BY PurchasedSubs DESC;"
```

```
cur.execute(sub_type_query)

results = cur.fetchall()

for sub_type, count in results:
    print(f"{sub_type} : {count}")
```

This is the result of the query:

```
Week : 10
Month : 10
Year : 9
Day : 4
```